

We are given a non-empty array A consisting of N integers ($N \geq 3$). Each element in the array can be either positive or negative.

The maximum product always comes from one of two cases:

- Product of the three largest values in the array
- Product of the two smallest values (negative) multiplied by the largest value

This is because multiplying two negative numbers produces a positive result, which can exceed the product of three positives

Examples:

Input Array	Expected Output	Reason
{2, 3, 4}	24	$2 \times 3 \times 4 = 24$
{6, 7, 8, 10}	560	$7 \times 8 \times 10 = 560$
{-10, -10, 5, 2}	500	$-10 \times -10 \times 5 = 500$ (two negatives)

2. Algorithm 1 - Sorting Approach (Non-Recursive)

Pseudo-code

```
function MaxProductOfThree(A):  
  
    BubbleSort(A)  
    n ← length(A)  
    product1 ← A[n-1] × A[n-2] × A[n-3]  
    product2 ← A[0] × A[1] × A[n-1]  
    return max(product1, product2)
```

```
function BubbleSort(A):  
    n ← length(A)  
    for i from 0 to n-2 do:  
        for j from 0 to n-i-2 do:  
            if A[j] > A[j+1] then  
                swap(A[j], A[j+1])  
            end if  
        end for  
    end for  
end function
```

Implementation (C++)

```
void bubbleSort(vector<int>& a) {
    int n = a.size();
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j+1]) {
                int temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
            }
        }
    }
}

long long maxProductSorting(vector<int> a) {
    bubbleSort(a);
    int n = a.size();
    long long product1 = 1LL * a[n-1] * a[n-2] * a[n-3];
    long long product2 = 1LL * a[0] * a[1] * a[n-1];
    return max(product1, product2);
}
```

Analysis & Complexity

Step-by-step analysis of the Sorting approach:

Step	Operation	Cost
1	Outer loop runs (n-1) times	O(n)
2	Inner loop runs (n-i-1) times per iteration	O(n)
3	Total comparisons = $n(n-1)/2$	O(n ²)
4	Calculate product1 = $a[n-1] \times a[n-2] \times a[n-3]$	O(1)
5	Calculate product2 = $a[0] \times a[1] \times a[n-1]$	O(1)
6	Return max(product1, product2)	O(1)

Time Complexity: O(n²) - dominated by Bubble Sort

Space Complexity: O(1) - As We uses a fixed number of variables

Algorithm 2 — Iterative Approach (Non-Recursive)

Pseudo-code

```
function MaxProductIterative(A):  
  
    max1  $\leftarrow -\infty$ , max2  $\leftarrow -\infty$ , max3  $\leftarrow -\infty$   
    min1  $\leftarrow +\infty$ , min2  $\leftarrow +\infty$   
  
    for each element x in A do:  
  
        if x > max1 then  
            max3  $\leftarrow$  max2; max2  $\leftarrow$  max1; max1  $\leftarrow$  x  
        else if x > max2 then  
            max3  $\leftarrow$  max2; max2  $\leftarrow$  x  
        else if x > max3 then  
            max3  $\leftarrow$  x  
        end if  
  
        if x < min1 then  
            min2  $\leftarrow$  min1; min1  $\leftarrow$  x  
        else if x < min2 then  
            min2  $\leftarrow$  x  
        end if  
    end for  
  
    product1  $\leftarrow$  max1  $\times$  max2  $\times$  max3  
    product2  $\leftarrow$  min1  $\times$  min2  $\times$  max1  
    return max(product1, product2)  
end function
```

Implementation (C++)

```
long long maxProductIterative(vector<int>& a) {  
  
    int max1 = INT_MIN, max2 = INT_MIN, max3 = INT_MIN;  
    int min1 = INT_MAX, min2 = INT_MAX;  
  
    for (int x : a) {  
  
        if (x > max1) { max3 = max2; max2 = max1; max1 = x; }  
        else if (x > max2) { max3 = max2; max2 = x; }  
        else if (x > max3) { max3 = x; }  
  
        if (x < min1) { min2 = min1; min1 = x; }  
    }  
}
```

```

        else if (x < min2) { min2 = x; }
    }

    // Case 1: three largest | Case 2: two smallest × largest

    long long p1 = 1LL * max1 * max2 * max3;
    long long p2 = 1LL * min1 * min2 * max1;
    return max(p1, p2);
}

```

Analysis & Complexity

Step-by-step analysis of the Iterative approach:

Step	Operation	Cost
1	Initialize 5 variables (max1,max2,max3,min1,min2)	O(1)
2	Loop over all N elements once	O(n)
3	Inside loop: at most 3 comparisons for max	O(1)
4	Inside loop: at most 2 comparisons for min	O(1)
5	Calculate $p1 = \text{max1} \times \text{max2} \times \text{max3}$	O(1)
6	Calculate $p2 = \text{min1} \times \text{min2} \times \text{max1}$	O(1)
7	Return $\text{max}(p1, p2)$	O(1)

Time Complexity: $O(n)$ - single pass through the array

Space Complexity: $O(1)$ - uses only 5 fixed variables regardless of input size

4. Algorithm 3 - Recursive Approach

Pseudo-code

```

function MaxProductRecursive(A, index, count, currentProduct):

    if count == 3:
        return currentProduct

    if index == length(A):
        return  $-\infty$ 

```

```

include = MaxProductRecursive(A, index+1, count+1, currentProduct × A[index])
exclude = MaxProductRecursive(A, index+1, count, currentProduct)

return max(include, exclude)
end function

```

Implementation (C++)

```

long long maxProductRecursive(vector<int>& a, int i, int count, long long product) {

    if (count == 3) return product;

    if (i == (int)a.size()) return LLONG_MIN;

    long long include = maxProductRecursive(a, i+1, count+1, product * a[i]);

    long long exclude = maxProductRecursive(a, i+1, count, product);

    return max(include, exclude);
}

```

Analysis & Complexity

Step-by-step analysis of the Recursive approach:

Step	Operation	Cost
1	Each element has 2 choices: include or exclude	2 branches
2	Tree depth = N (one level per element)	N levels
3	Total nodes in recursion tree = 2^N	$O(2^N)$
4	Each node does $O(1)$ work (compare + multiply)	$O(1)$

5	Total work = $2^N \times O(1)$	$O(2^N)$
6	Max call stack depth = N	$O(N)$ space

Time Complexity: $O(2^n)$ — tries all possible combinations

Space Complexity: $O(n)$ — recursive call stack depth equals array size

5. Comparison Between Algorithms

	Sorting (Bubble)	Iterative	Recursive
Time Complexity	$O(n^2)$	$O(n)$	$O(2^n)$
Space Complexity	$O(1)$	$O(1)$	$O(n)$
Passes over array	Multiple (n-1)	Single pass	All combinations
Handles negatives	✓ Yes	✓ Yes	✓ Yes
Practical for large n	Moderate	✓ Best	✗ Very slow
Code simplicity	Medium	Medium	Simple logic

- Iterative $O(n)$ — Best: single pass, no extra memory
- Sorting $O(n^2)$ — Good: simple but slower due to Bubble Sort
- Recursive $O(2^n)$ — Worst: impractical for large arrays

Conclusion: The Iterative approach is the optimal solution. It scans the array only once, uses constant memory, and correctly handles all edge cases including negative numbers and zeros.

6. Test Cases & Output

Array	Sorting	Iterative	Recursive	Expected
{2, 3, 4}	24	24	24	24
{6, 7, 8, 10}	560	560	560	560

{-10, -10, 5, 2}	500	500	500	500
------------------	-----	-----	-----	-----